Documentación del Proyecto Hotel

Configuración y Scripts de Ejecución

HotelApp – Asistente Virtual (Android, Kotlin + Jetpack Compose)

1. Descripción general

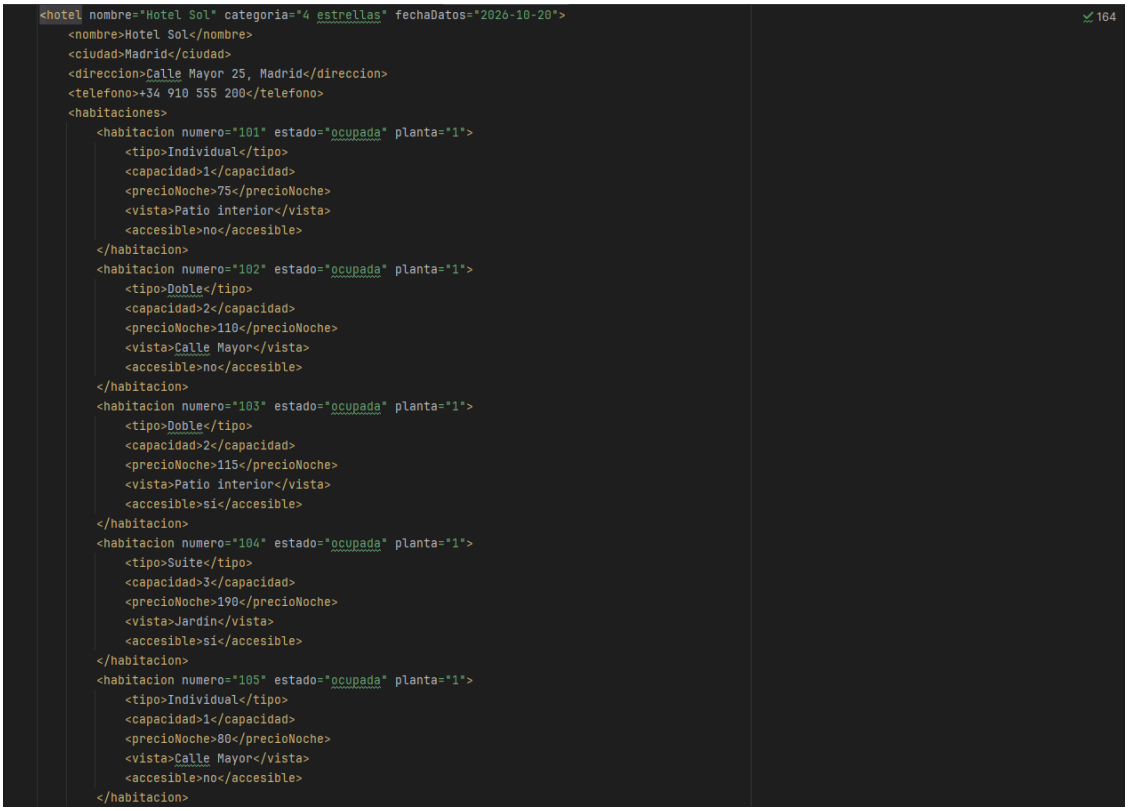
HotelApp es una aplicación Android con un asistente virtual en formato chat para consultar y reservar habitaciones de hotel. El usuario puede escribir su petición o dictarla por voz. Las habitaciones disponibles se leen de un archivo XML y se muestran en el chat como tarjetas con botón de reserva.

2. Datos de origen: archivo XML

Las habitaciones se definen en un XML cuyo elemento raíz es <hotel>, con los atributos nombre ("Hotel Sol"), categoría ("4 estrellas") y fechaDatos ("2026-10-20"). Dentro contiene los datos del hotel (nombre, ciudad, dirección y teléfono) y un bloque <habitaciones>.

Cada <habitacion> tiene los atributos numero, estado y planta, y estos elementos hijos:

- tipo: Individual, Doble o Suite.
- capacidad: número de personas.
- precioNoche: precio por noche en euros.
- vista: Patio interior, Calle Mayor, Jardín, etc.
- accesible: sí / no.



```xml
<hotel nombre="Hotel Sol" categoria="4 estrellas" fechaDatos="2026-10-20">
    <nombre>Hotel Sol</nombre>
    <ciudad>Madrid</ciudad>
    <direccion>Calle Mayor 25, Madrid</direccion>
    <telefono>+34 910 555 200</telefono>
    <habitaciones>
        <habitacion numero="101" estado="ocupada" planta="1">
            <tipo>Individual</tipo>
            <capacidad>1</capacidad>
            <precioNoche>75</precioNoche>
            <vista>Patio interior</vista>
            <accesible>no</accesible>
        </habitacion>
        <habitacion numero="102" estado="ocupada" planta="1">
            <tipo>Doble</tipo>
            <capacidad>2</capacidad>
            <precioNoche>110</precioNoche>
            <vista>Calle Mayor</vista>
            <accesible>no</accesible>
        </habitacion>
        <habitacion numero="103" estado="ocupada" planta="1">
            <tipo>Doble</tipo>
            <capacidad>2</capacidad>
            <precioNoche>115</precioNoche>
            <vista>Patio interior</vista>
            <accesible>si</accesible>
        </habitacion>
        <habitacion numero="104" estado="ocupada" planta="1">
            <tipo>Suite</tipo>
            <capacidad>3</capacidad>
            <precioNoche>190</precioNoche>
            <vista>Jardín</vista>
            <accesible>si</accesible>
        </habitacion>
        <habitacion numero="105" estado="ocupada" planta="1">
            <tipo>Individual</tipo>
            <capacidad>1</capacidad>
            <precioNoche>80</precioNoche>
            <vista>Calle Mayor</vista>
            <accesible>no</accesible>
        </habitacion>
```

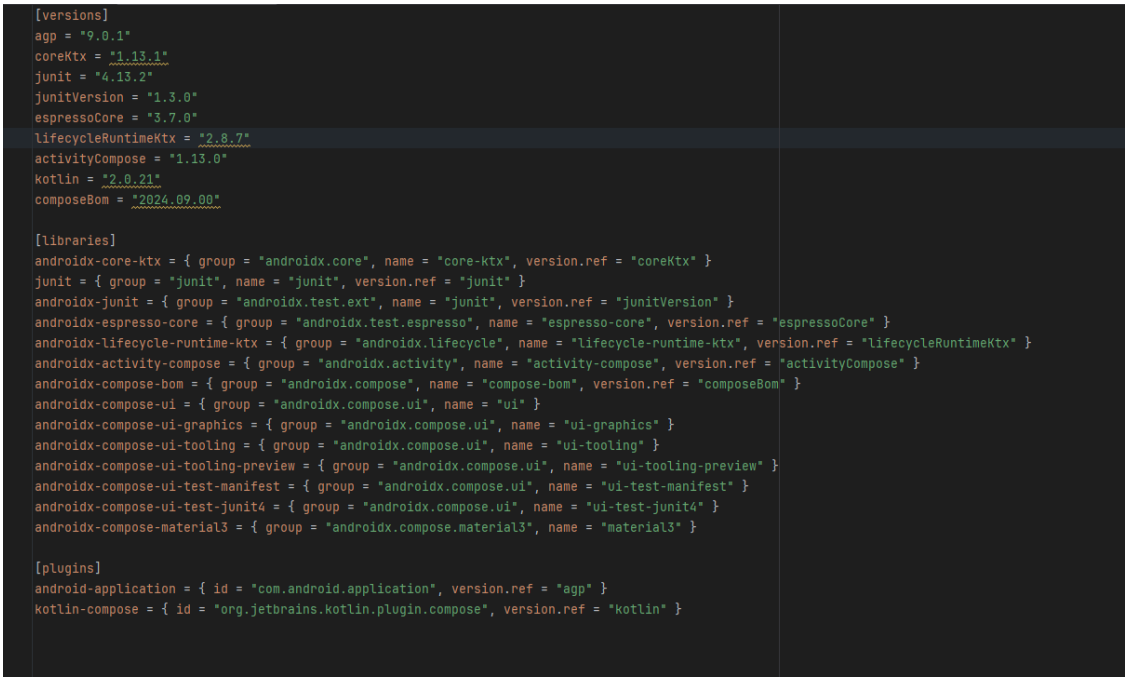
Figura 1. Estructura del XML de habitaciones (Hotel Sol, Madrid).

3. Configuración del proyecto

3.1 Catálogo de versiones (libs.versions.toml)

Las dependencias se gestionan con el catálogo de versiones de Gradle, dividido en tres bloques:

- [versions]: agp 9.0.1, coreKtx 1.13.1, junit 4.13.2, junitVersion 1.3.0, espressoCore 3.7.0, lifecycleRuntimeKtx 2.8.7, activityCompose 1.13.0, kotlin 2.0.21 y composeBom 2024.09.00.
- [libraries]: core-ktx, junit, androidx-junit, espresso-core, lifecycle-runtime-ktx, activity-compose, compose-bom y los módulos de Compose (ui, ui-graphics, ui-tooling, ui-tooling-preview, ui-test-manifest, ui-test-junit4 y material3).
- [plugins]: com.android.application (versión agp) y org.jetbrains.kotlin.plugin.compose (versión kotlin).



```
[versions]
agp = "9.0.1"
coreKtx = "1.13.1"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
lifecycleRuntimeKtx = "2.8.7"
activityCompose = "1.13.0"
kotlin = "2.0.21"
composeBom = "2024.09.00"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

Figura 2. Archivo libs.versions.toml con versiones, librerías y plugins.

4. Código principal (MainActivity.kt)

La actividad principal define dos modelos de datos y arranca la interfaz Compose:

- Habitacion: modelo de cada habitación del XML (numero, estado, planta, tipo, capacidad, precioNoche, vista y accesible).
- ChatMessage: modelo de cada mensaje del chat, con text, isUser y una habitacionCard opcional para mostrar la tarjeta.
- MainActivity: en onCreate activa enableEdgeToEdge(), carga las habitaciones con cargarHabitacionesDesdeXML() desde reservas.xml en assets y muestra HotelChatScreen dentro de HotelTheme.

El XML se procesa con DocumentBuilderFactory (javax.xml.parsers) y las clases Element y Node de org.w3c.dom.

```
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import es.ejemplo.android.hotel.ui.theme.HotelTheme
import org.w3c.dom.Element
import org.w3c.dom.Node
import javax.xml.parsers.DocumentBuilderFactory

// Modelo de datos para las habitaciones del XML
6 Usages
data class Habitacion(
    val numero: String,
    val estado: String,
    val planta: String,
    val tipo: String,
    val capacidad: Int,
    val precioNoche: Double,
    val vista: String,
    val accesible: String
)

// Modelo para cada mensaje en la conversación del chat
8 Usages
data class ChatMessage(
    val text: String,
    val isUser: Boolean,
    val habitacionCard: Habitacion? = null
)

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        // Cargar habitaciones desde reservas.xml en assets
        val habitaciones = cargarHabitacionesDesdeXML()

        setContent {
            HotelTheme {
                HotelChatScreen(habitaciones = habitaciones)
            }
```

Figura 3. Modelos de datos y onCreate de MainActivity.

5. Ejecución de la aplicación

La aplicación se ejecuta desde Android Studio con el botón Run de MainActivity, sobre un emulador o un dispositivo Android. Al abrirla muestra el asistente virtual con el saludo inicial. Las capturas siguientes muestran el flujo de uso.





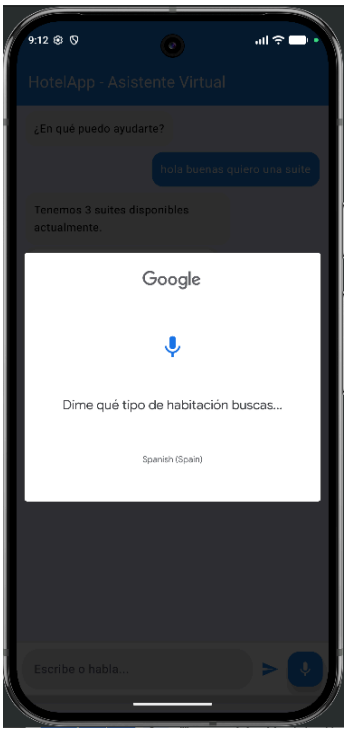


Figura 4. Pantalla inicial *Figura 5. Búsqueda de una suite* *Figura 6. Entrada por voz*

5.1 Pantalla inicial

La barra superior muestra "HotelApp - Asistente Virtual". El asistente saluda con "¿En qué puedo ayudarte?". Abajo hay un campo de texto ("Escribe o habla..."), un botón de enviar y un botón de micrófono.

5.2 Consulta por texto

El usuario escribe "hola buenas quiero una suite". El asistente responde que hay 3 suites disponibles y muestra una tarjeta con la Habitación 115 (Suite, 3 personas, 210,0 € por noche) y el botón "Reservar".

5.3 Consulta por voz

Al pulsar el micrófono se abre el reconocimiento de voz de Google, configurado en español de España, con el mensaje "Dime qué tipo de habitación buscas...". El texto reconocido se envía al chat como si se hubiera escrito.